설비가 실제로 한 일만 실적이 됩니다

가상 포장라인 · 실제 암호화 OPC UA · 웹 HMI · MES 작업지시와 정산

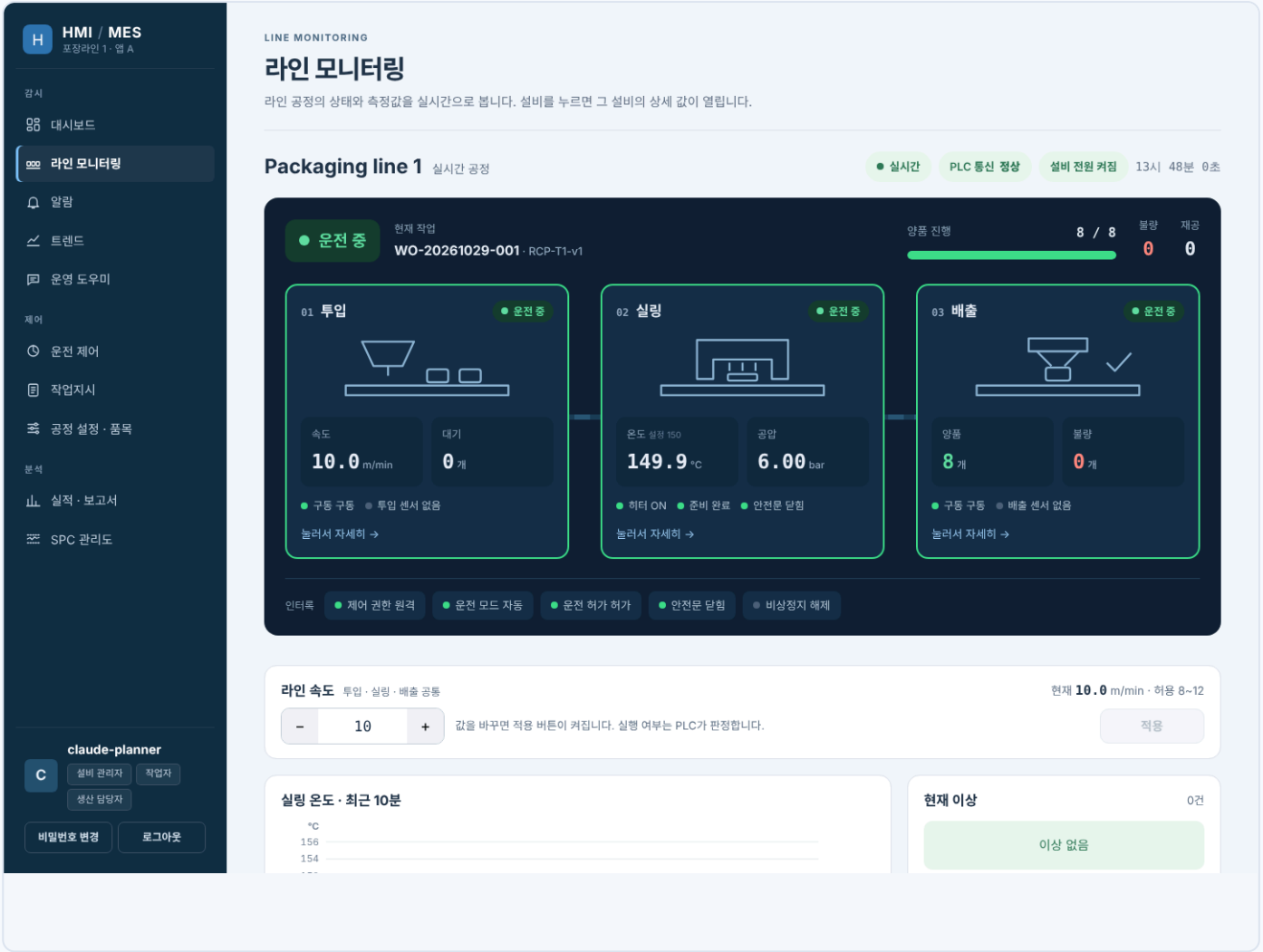
HMI MES.

가상 PLC가 판정하고,
OPC UA로 수집하고,
정산이 끝나야 다음 작업.

1인 프로젝트 | AI 코딩 에이전트 활용 | 공개 데모

작성자 전우석 · 개발 기간 9일

2026.10.20 ~ 10.28 · 커밋 264개
기획 방향·운영 규칙 결정, 화면 검수와 수정 요구, 개발 조율
Codex · Claude Code를 활용한 구현과 검증



공개 배포 hmi-mes.vercel.app · 2026-10-29 실제 운전 중 라인 모니터링. 값은 설비 앱이 OPC UA로 보낸 수집값입니다.

“버튼을 눌렀다”와 “설비가 움직였다”는 다릅니다

설비 현장에서 화면이 거짓말하기 쉬운 지점을 처리 규칙으로 정했습니다.

명령의 결과를 모르는 순간

요청 접수와 실행 결과를 분리하고, 응답이 끊기면 같은 명령 ID로 조회만 합니다.

통신이 끊긴 동안의 값

측정값마다 품질과 시각을 보관합니다. 불명을 0이나 정상으로 바꾸지 않습니다.

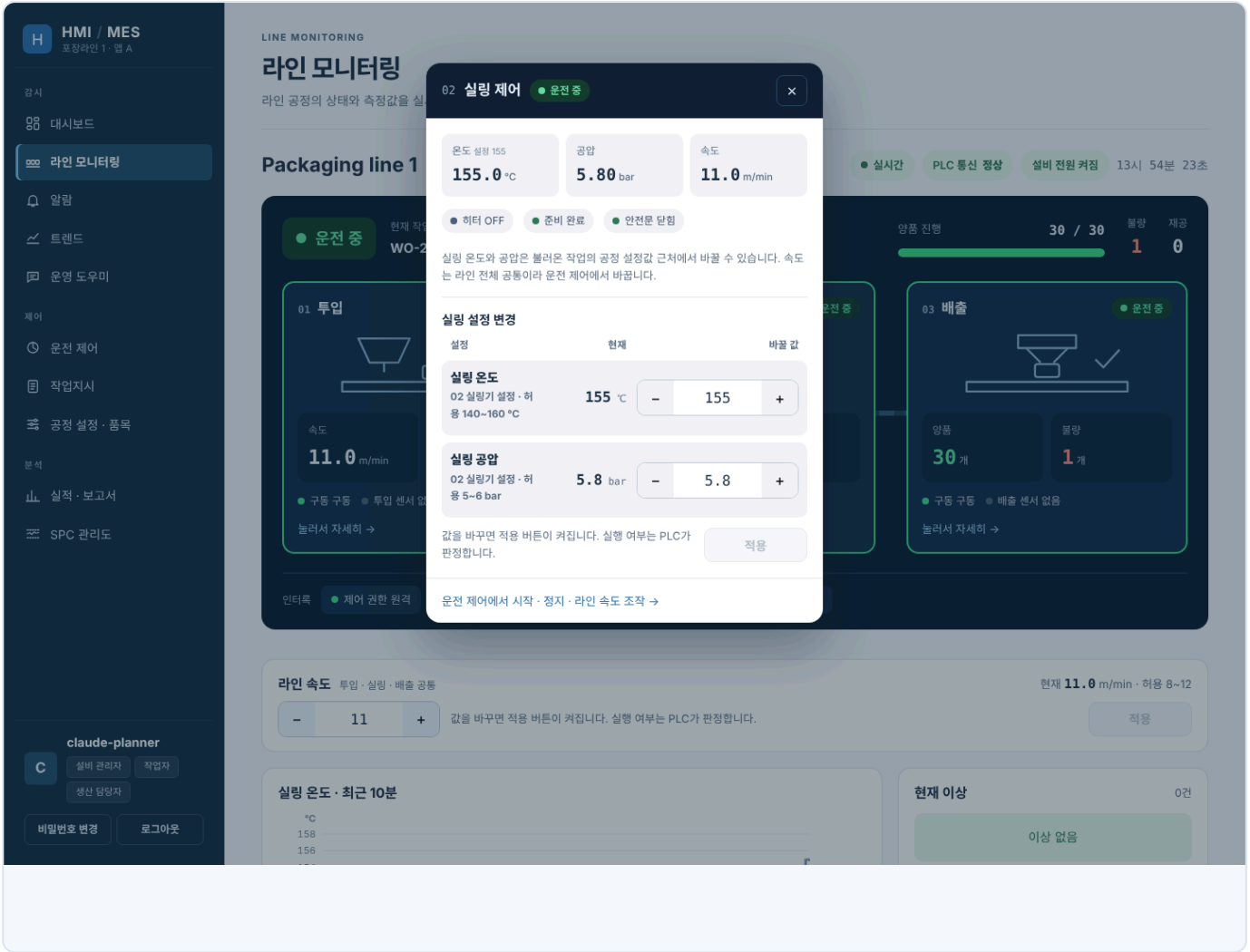
재전송과 재시작

생산 사건마다 고유 ID와 원래 실행 ID를 붙여 실적이 두 번 잡히지 않게 합니다.

작업 전환

재공을 정리하고 설비 최종 수량과 대조한 뒤에만 다음 작업을 연결합니다.

> KULS 공개 HMI 구성예시 기준 · 현장설비 → Master PLC → OPC 서버 → HMI·MES 서버 → 모니터링의 각 계층을 직접 구현



실링기 상세 창. 운전 중 바꾼 온도 155 °C · 공압 5.8 bar · 속도 11 m/min이 설비 수집값으로 돌아온 화면입니다.

두 앱은 DB를 공유하지 않고 OPC UA로만 연결합니다

앱 A 운영 시스템 · 앱 B 설비 시뮬레이터 · 게이트웨이 · PostgreSQL



설명용 구조도. 운전 허용 여부와 실행 결과는 앱 B의 가상 PLC가 최종 판정합니다.



운전 제어의 설비 전원 단계. 원격 실행 환경의 설비 앱을 켜고 끕니다.

계층	선택과 목적
설비	asyncua OPC UA 서버 · 가상 PLC · SQLite 원자 저장
연동	OPC UA 클라이언트 · 디스크 버퍼 · 재전송
운영	Python API · PostgreSQL · Alembic
화면	React · TypeScript · Vite
데모	Vercel + Neon 무료 요금제 · 설비는 임시 클라우드 실행

전원부터 정산까지 다섯 단계로 안내합니다

작업자는 지금 단계에서 할 일만 봅니다. 다음 단계로 넘어갈 조건은 서버와 PLC가 판정합니다.

설비 전원

설비 앱·게이트웨이 기동, 통신 정상 확인

작업 준비

배포된 작업지시를 한 번에 불러오기

예열

실링 온도 도달을 PLC가 판정, 자동 시작 선택

생산

양품 / 목표, 일시정지·재개, 운전 중 설정 변경

종료·정산

재공 정리 → 최종 수량 대조 → 다음 작업

촬영 시나리오: 작업지시 등록·배포 → 불러오기 → 예열 → 생산 → 일시정지·재개 → 종료 → 정산 → 기존 작업지시 재실행



운전 중인 운전 제어 화면. 단계 바, 현재 단계 패널, 라인 조건이 함께 보입니다.

속도·온도·공압을 바꾸되, 허용 여부는 PLC가 정합니다

화면은 요청만 보냅니다. 범위 밖 요청은 서버가 먼저 막고, 최종 판정은 PLC가 합니다.

레시피 기준 허용 폭

라인 속도 ±20% · 실링 온도 ±10 °C · 실링 공압 ±0.5 bar. 기준은 작업에 연결된 공정 설정 버전입니다.

PLC 최종 판정

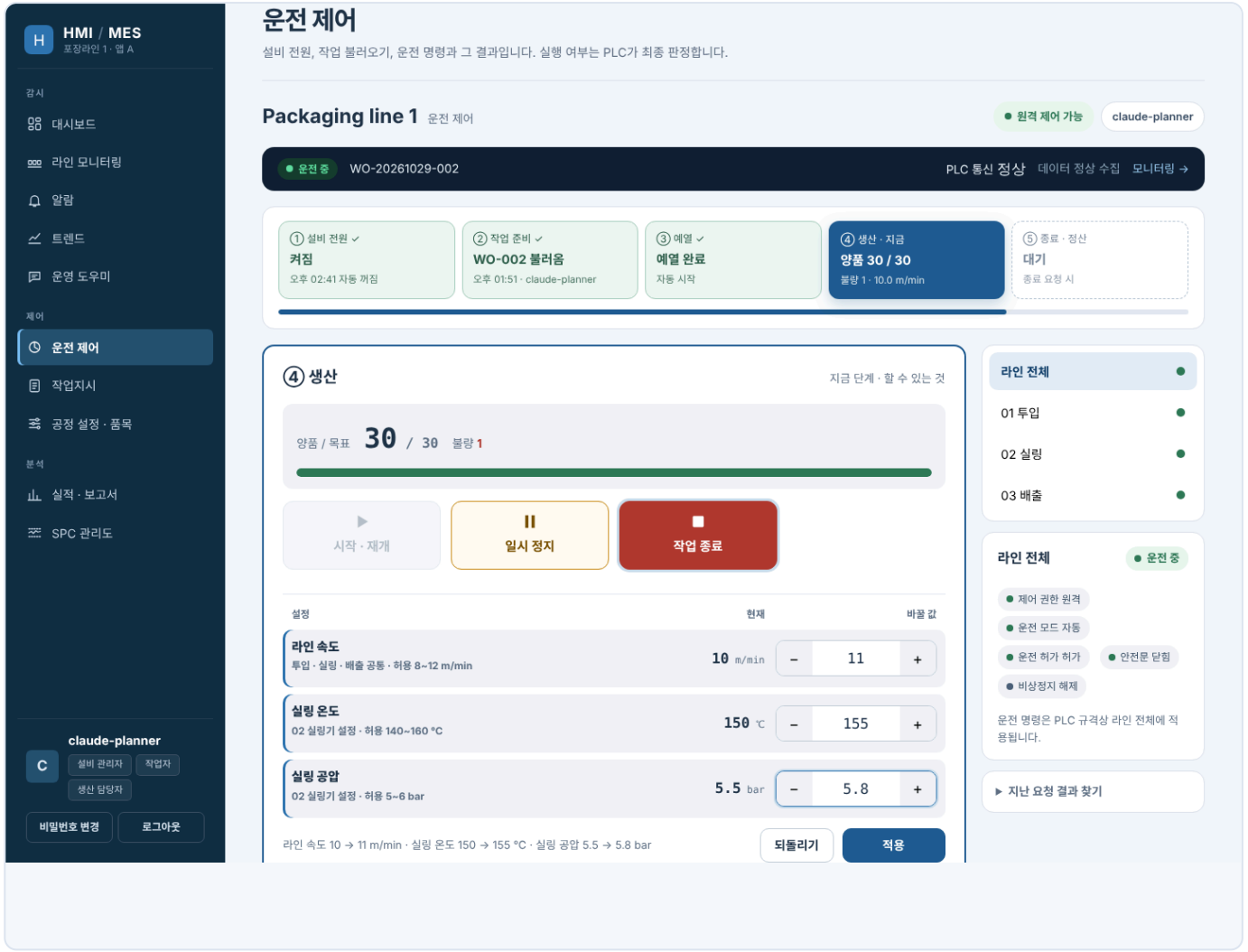
실링 공압 4.5~6 bar 범위 밖이면 PLC가 거절합니다. 화면은 거절 사유를 그대로 표시합니다.

컨베이어는 하나의 라인 속도

투입·실링·배출이 같은 라인 속도를 공유합니다. 모니터링 화면에서도 같은 값을 조절합니다.

결과까지 추적

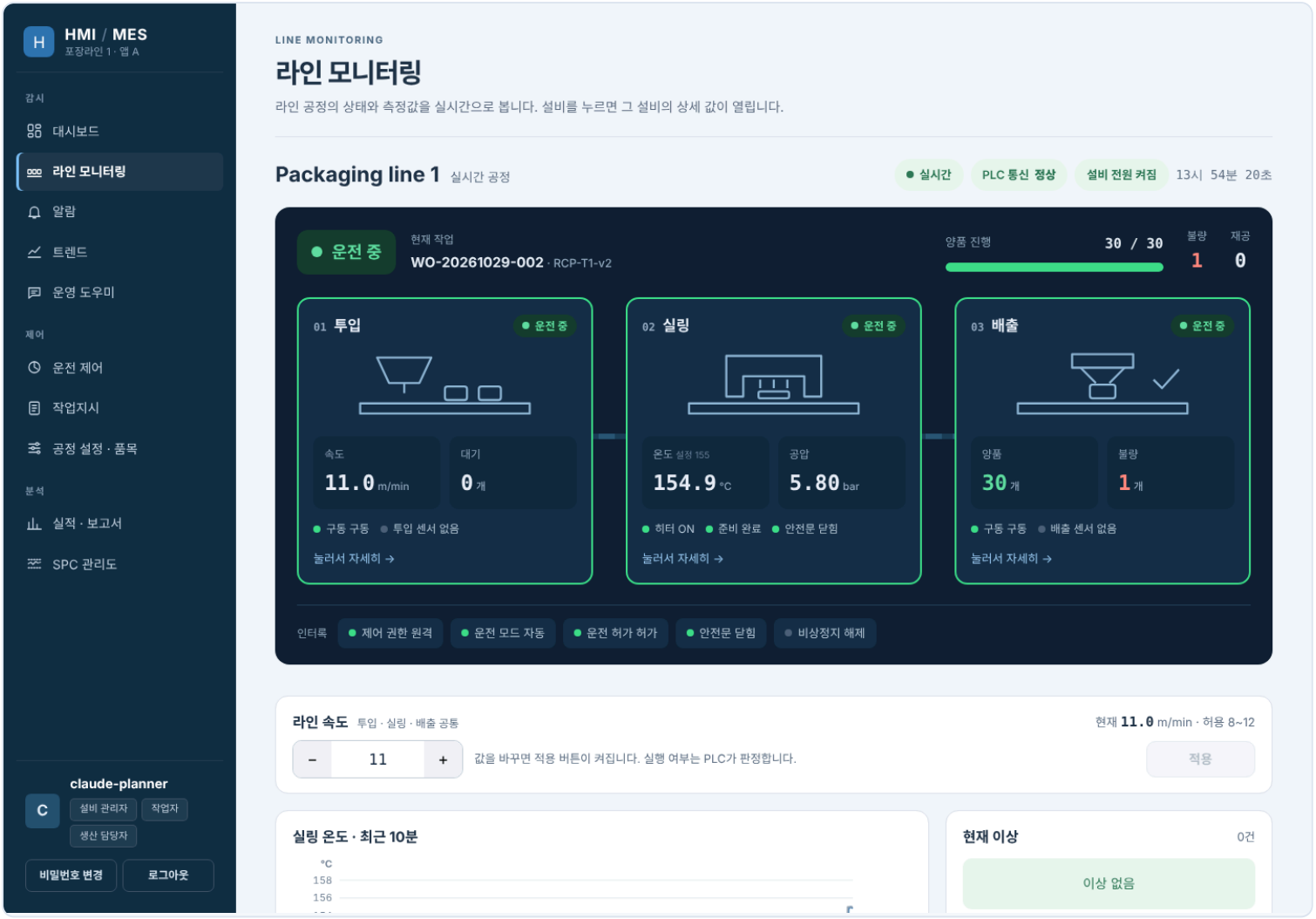
변경 요청도 일반 명령과 같이 원래 명령 ID로 접수·결과를 확인하고, 실행별 변경 기록을 남깁니다.



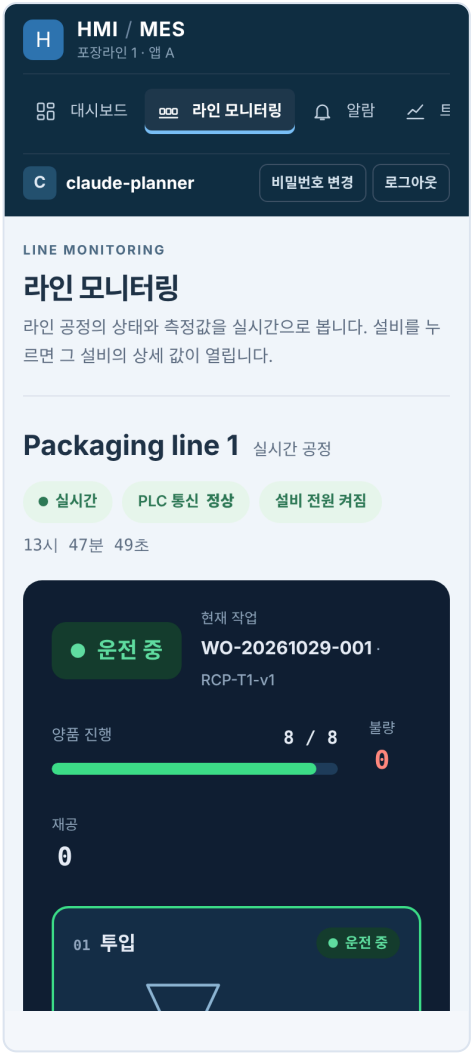
공압이 들어간 공정 설정 v2로 운전 중. 바꿀 값과 허용 범위, 적용 전 변경 내용을 함께 보여 줍니다.

세 설비의 상태와 데이터 품질을 한 화면에서 봅니다

공정도 중심 화면 · 설비별 상세 창 · 휴대폰 폭 대응



라인 속도 카드와 설비 상태. 데스크톱 1440px 폭에서 촬영했습니다.



같은 시각의 휴대폰 폭(390px) 화면입니다.

작업지시는 한 번 만들고, 여러 번 실행합니다

작업지시는 계획이고, 실행은 회차입니다. 실적은 회차마다 원래 실행 ID에 귀속합니다.

등록 · 배포 · 수정 · 취소

건별로 관리하고 누가 언제 무엇을 했는지 진행 기록에 남깁니다.

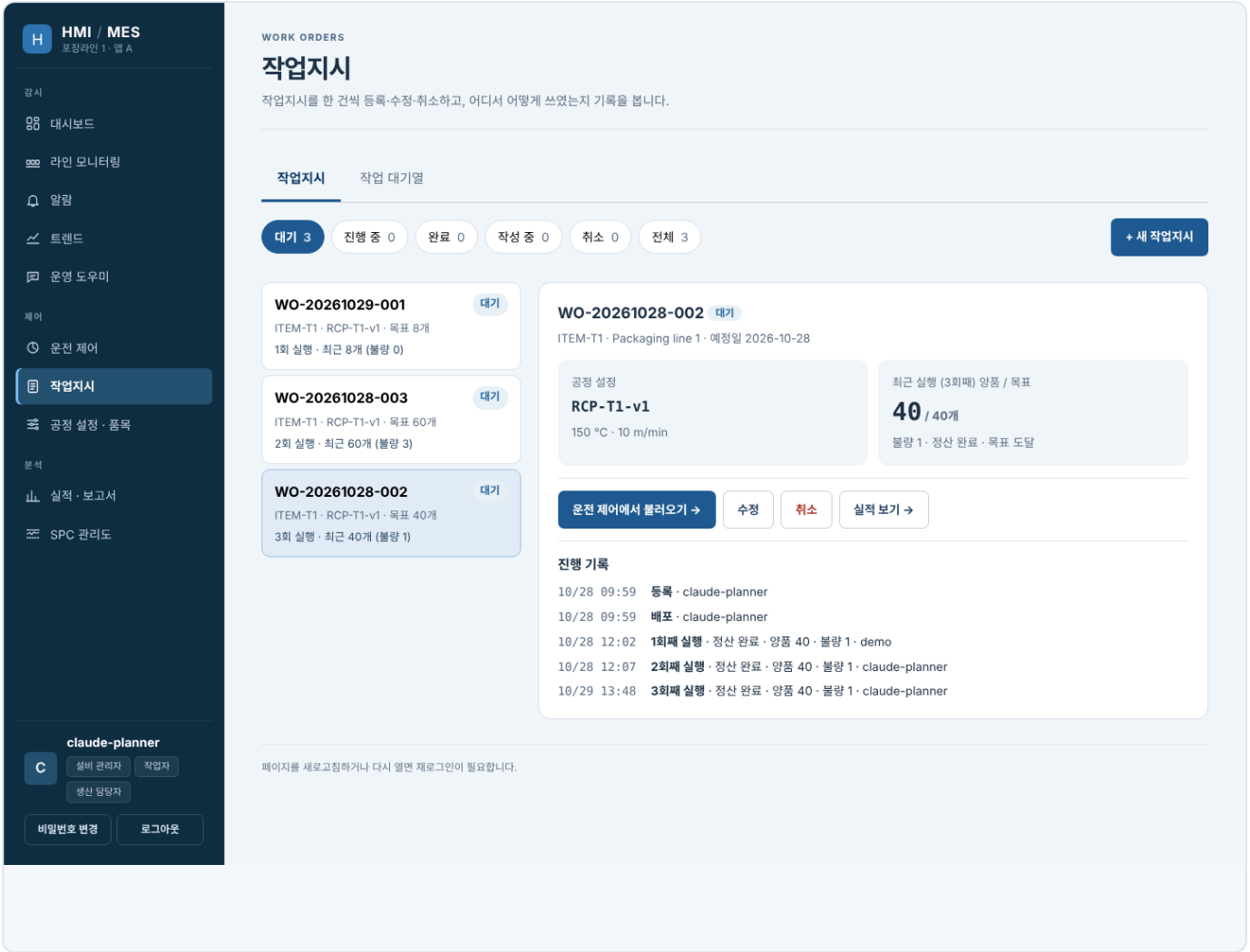
재실행

같은 작업지시를 다시 불러오면 새 실행 ID를 발급합니다. 이전 회차의 실적은 그대로 보존됩니다.

공정 설정 버전

온도·속도·공압을 버전으로 고정합니다. 배포 후에는 수정하지 않고 새 버전을 만듭니다.

> 이번 촬영: 기존 작업지시 WO-002의 3회째 실행과, 공압 5.5 bar를 넣은 공정 설정 v2로 새 작업지시 실행



기존 작업지시를 다시 실행한 뒤의 진행 기록. 실행 회차와 회차별 결과가 쌓입니다.

응답이 끊겨도 새 ID로 다시 실행하지 않습니다

접수 · 실행 · 결과 불명을 구분합니다. 확인되지 않은 명령은 원래 ID로만 조회합니다.

접수와 결과 분리

API는 명령을 원장에 예약하고, 게이트웨이가 PLC에 전달한 뒤 결과를 따로 조회해 보고합니다.

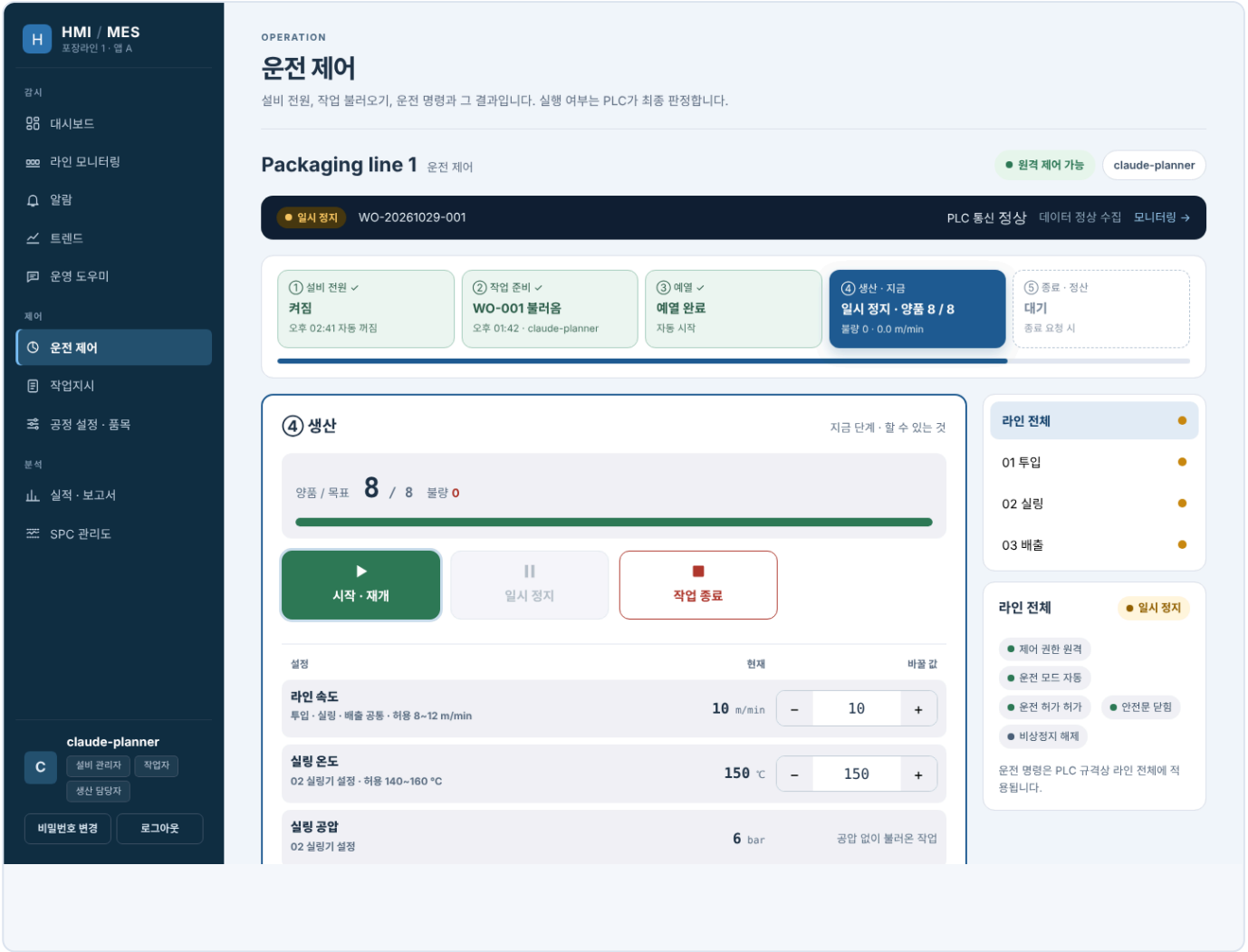
결과 불명은 불명으로

만료나 통신 단절로 결과를 모르면 OUTCOME_UNKNOWN으로 남기고, 같은 명령 ID의 조회만 허용합니다.

권한은 첫 제어 경로부터

세션·역할·CSRF·Origin을 검사합니다. 조회 전용 사용자의 제어 요청은 서버가 거절합니다.

> 촬영 중 실제 사례: 설정 변경 1건과 종료 1건의 응답이 확인되지 않아 '결과 불명'으로 남았습니다. 설비 값과 정산은 나중에 확인됐지만, 화면은 해당 요청을 성공으로 바꾸지 않았습니다. 응답이 끊긴 원인은 아직 조사하지 않았습니다.



일시정지 명령이 성공한 직후. 명령 결과와 라인 상태가 함께 표시됩니다.

같은 사건이 두 번 와도 실적은 한 번만 반영합니다

생산 사건 · 중복 제거 · 처리 커서를 한 트랜잭션에 저장하고, 정산으로 설비 수량과 대조합니다.

사건 단위 멱등

생산 사건마다 고유 ID와 원래 실행 ID를 붙입니다. 이미 받은 사건은 다시 반영하지 않습니다.

커밋 ACK 후 버퍼 삭제

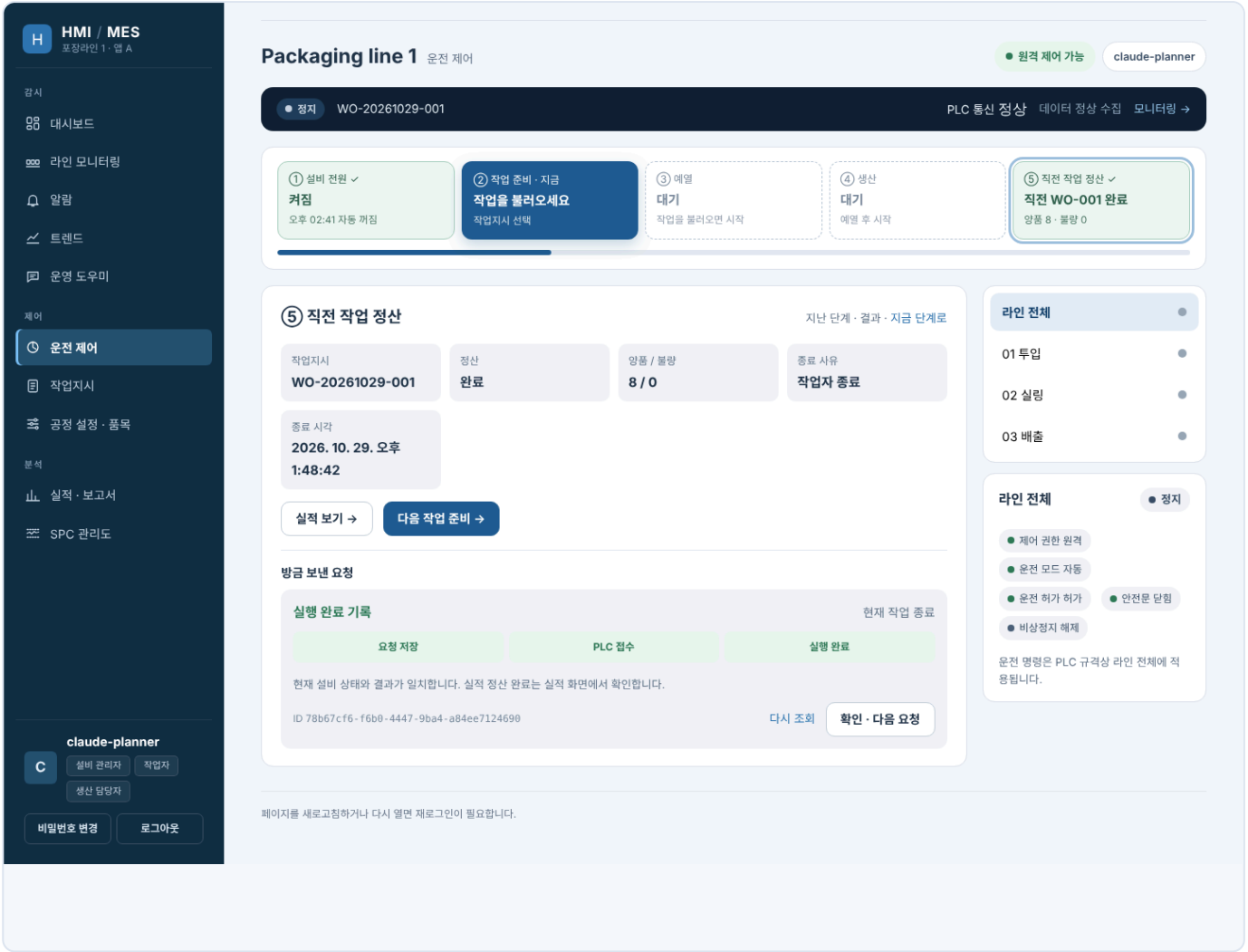
게이트웨이는 DB 커밋 응답을 받은 사건만 버퍼에서 지웁니다. 응답이 끊기면 같은 배치를 다시 보냅니다.

누락은 누락으로

수신 순번에 빈 구간이 생기면 정산 완료로 바꾸지 않고 확인 필요로 남깁니다.

정산 후 다음 작업

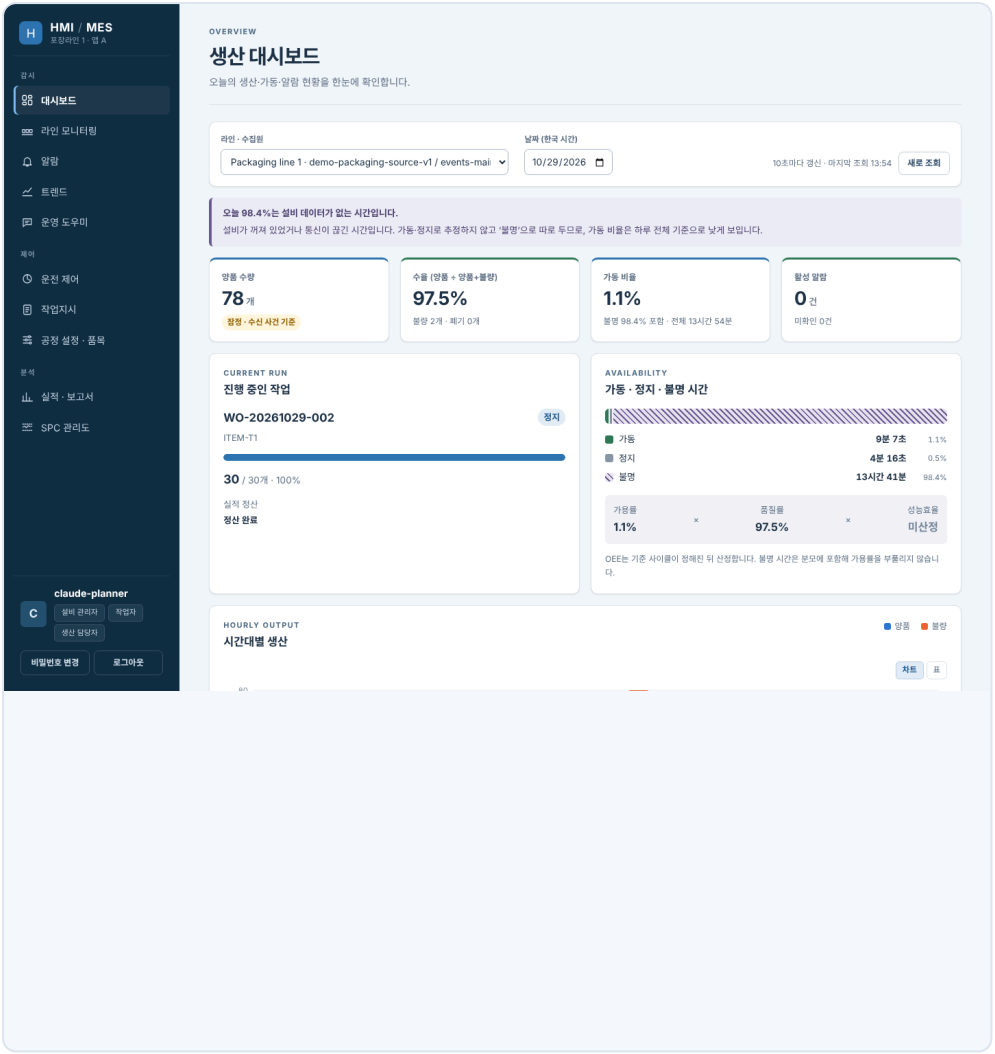
종료 요청 → 신규 투입 중단 → 재공 정리 → 최종 요약 대조가 끝나야 다음 작업을 불러옵니다.



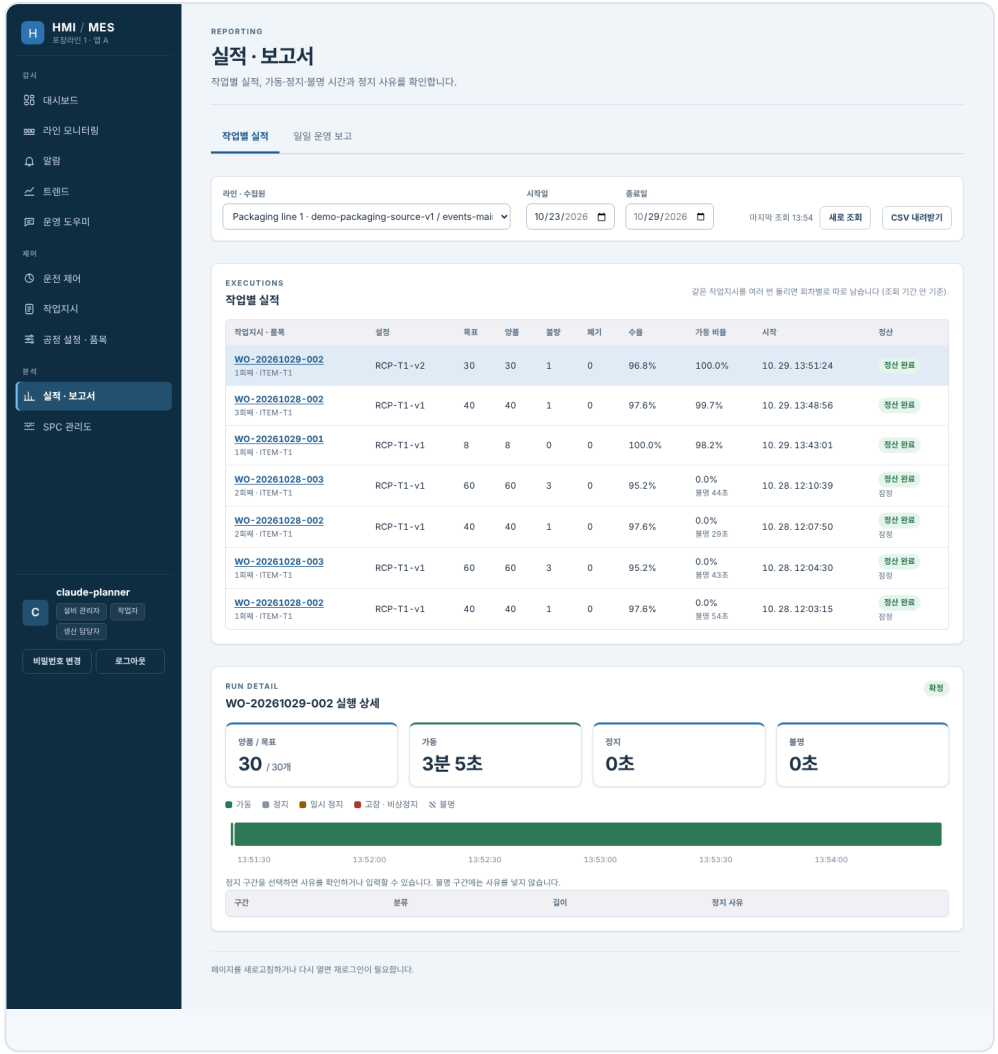
종료 후 정산 결과. 직전 작업의 양품·불량이 확정된 뒤 작업 준비 단계로 돌아갑니다.

하루 생산과 실행별 실적을 같은 근거로 집계합니다

가동·정지·불명 시간을 나눠 보여 줍니다. 근거가 없는 시간은 가동으로 계산하지 않습니다.



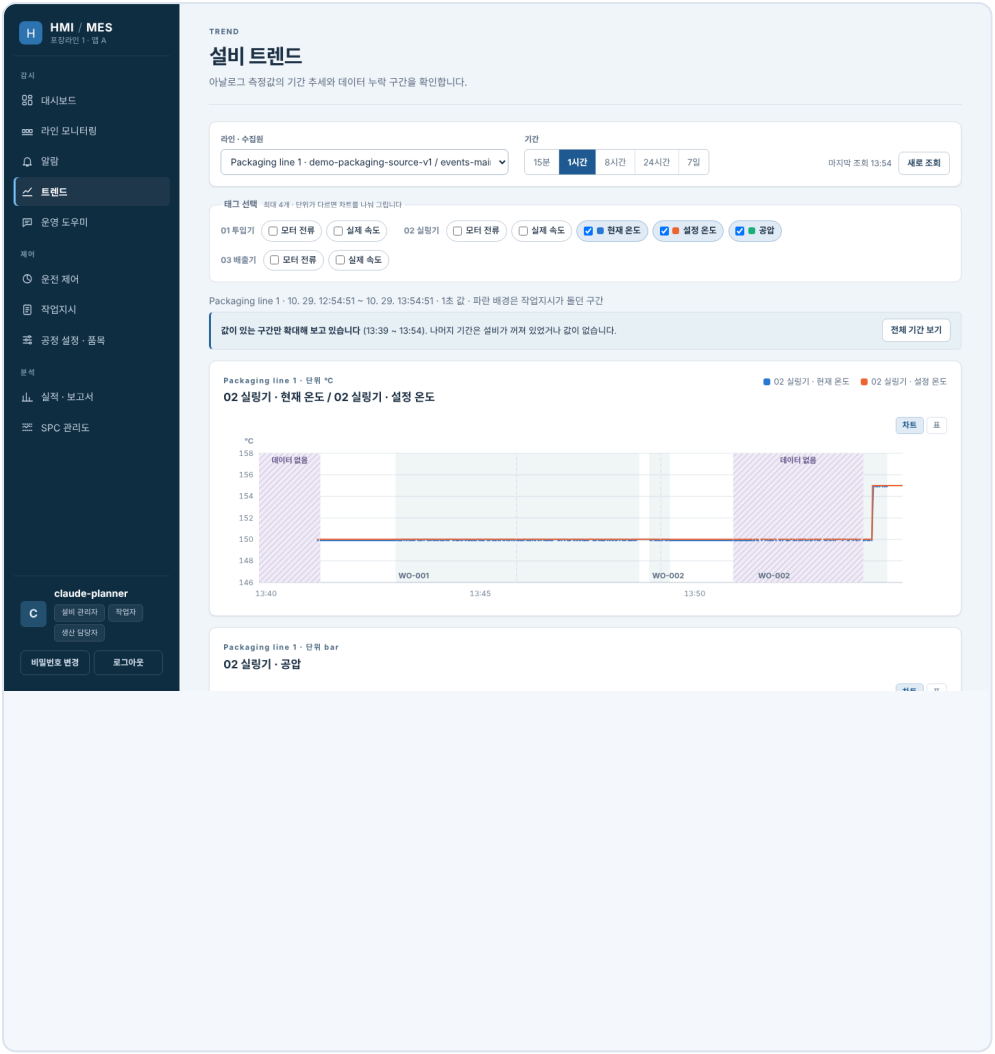
촬영 당일 대시보드. 설비가 꺼져 있던 시간은 가동·정지로 추정하지 않고 '불명'으로 남겨 가동 비율이 낮게 보입니다.



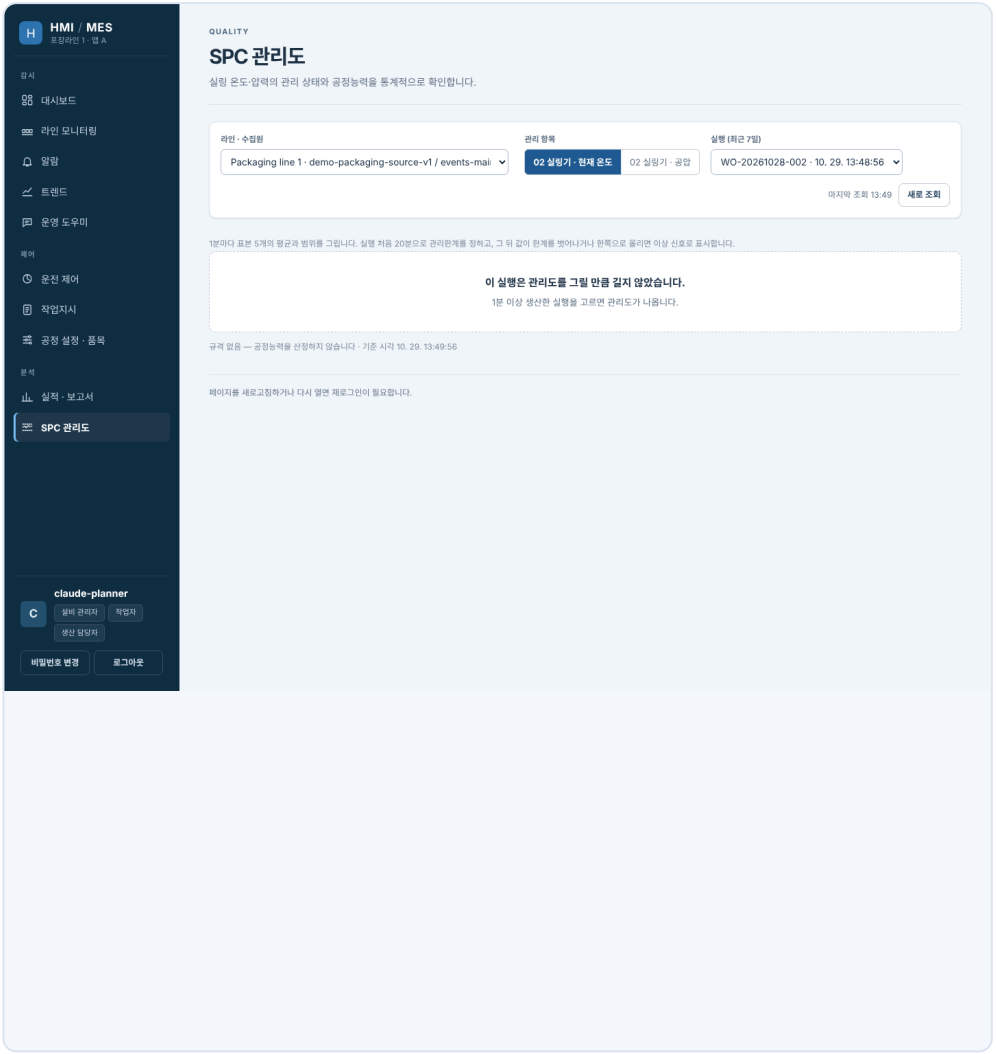
실행별 보고서. 같은 작업지시의 회차를 따로 집계하고, 일시정지 구간까지 시간 막대에 표시합니다.

공정값의 흐름과 관리 상태를 확인합니다

태그별 기간 조회와 실링 온도·공압 관리도. 데이터가 부족하면 계산하지 않고 그렇게 표시합니다.



실링 온도 트렌드. 운전 중 설정을 150 → 155 °C로 바꾼 계단과 설비가 꺼진 '데이터 없음' 구간이 보입니다.



SPC 관리도. 이번 촬영은 실행마다 몇 분짜리라, 20분 기준 관리한계를 계산하지 않고 그 사실을 표시했습니다.

통과한 범위와 남은 기준을 구분했습니다

시험 개수 · 공개 배포 운전 · 수락 시험 · 장시간 성능을 하나의 완료 주장으로 합치지 않습니다.

2,354
Python 시험 · 전체 로컬 검사(10-23)

28
DB 마이그레이션

검증	결과와 범위
전체 로컬 검사	2026-10-23 통과 · Python 2,354 · 웹 91 · 타입·린트·빌드
실제 경로 검사	브라우저 → API → PostgreSQL → 게이트웨이 → OPC UA → 정산 PASS (10-23)
공개 배포 운전	2026-10-29 이 문서의 촬영 사이클을 실제 운전으로 수행
10-24 이후 변경	변경 범위의 단위·계약·화면 시험만 실행. 전체 검사 재실행 전

시험 개수는 자동화 시험의 실행 수이며 수작업 작성량을 뜻하지 않습니다.

남은 기준 (M5 운영 단계 미완료)

수락 시험 T01~T12는 17개로 나눠 통과했지만 한 번에 묶은 실행은 하지 않았습니다. **30분 통신 단절 후 5분 안에 복구하는 시험은 실패**했습니다. 7일 이력 성능은 미실행, 8시간 연속 운전은 범위에서 뺐습니다.

공개 데모의 제약

공개 배포에는 알람 생성이 연결되지 않았습니다(설치된 규칙 0개). 무료 DB 한도를 넘겨 새 프로젝트로 이전했고, 시험 데이터 삭제가 수집 순번 검증을 깨뜨려 수집 스트림을 초기화했습니다. 작업지시·계정·공정 설정은 보존했습니다.

설비는 소프트웨어 시뮬레이터이며 실물 PLC·현장 설비 검증이 아닙니다. 상용 현장 운영 성과로 표시하지 않습니다.

화면을 직접 써 보며 누락을 찾고, 규칙을 정했습니다

기준 문서 → 작은 작업 단위 → 로컬 검증 → 공개 배포 확인. 미실행·실패는 통과로 기록하지 않았습니다.

직접 발견 · 수정 요구

① 작업할 때마다 작업지시를 새로 만들어야 했다

발견 한 번 실행한 작업지시는 다시 불러올 수 없어, 같은 제품을 또 만들려면 지시서를 새로 등록해야 했습니다.

판단 작업지시는 유지하는 계획이고 실행은 회차라고 정했습니다. 실적은 회차별 실행 ID에 귀속합니다.

구현·검증 작업지시–실행 연결의 1:1 제약을 풀고, 대기열 중복 방지는 대기 중인 항목에만 적용했습니다. 진행 기록에 회차 목록을 보여 줍니다.

> **설계 근거** 회차를 나누지 않으면 두 번째 실행의 수량이 첫 실행에 섞입니다.

직접 발견 · 수정 요구

② 제어 화면인데 온도·공압을 못 바꿨다

발견 운전 중 바꿀 수 있는 값이 속도뿐이었고, 공압은 공정 설정에도 없었습니다.

판단 화면에서 바꿀 수 있게 하되, 허용 폭은 공정 설정 기준으로 서버가 먼저 막고 PLC가 최종 판정하도록 정했습니다.

구현·검증 공정 설정에 공압 항목을 추가하고, PLC 공압 범위와 운전 중 변경 허용 폭을 시험으로 고정했습니다.

> **설계 근거** 화면 검사만으로는 다른 경로의 요청을 막지 못합니다. 판정은 설비 쪽에 남깁니다.

본인 기여: 기획 방향과 운영 규칙 결정, 화면 검수와 문제 발견, 수정 요구, 개발 조율. **AI 지원:** 코드 작성·시험 생성·문서화(Codex · Claude Code). 모든 코드를 수작업 작성했다고 표현하지 않습니다.

근거: AGENTS.md 작업 규칙 · docs/DEVELOPMENT_STATUS.md · 마이그레이션 0027(공압) · 0028(작업지시 재사용)

결과를 모르면 다시 보내지 않고, 원래 ID로 묻습니다

실제 코드 발췌 · CommandWorker.run_once · apps/gateway/commands.py

고정 소스 L161~L197 발췌 · 시계·통신 상태 확인 4줄 생략

```
command_id = work.request.command_id
submitted = False
if isinstance(work, SubmitWork):
    now = self.clock()
    if now >= work.request.expires_at:
        # The API already reserved OUTCOME_UNKNOWN; only a later QUERY can resolve it.
        return CommandWorkResult("deferred", command_id)
    try:
        response = await self.opc.submit_command(work.request)
        submission = CommandSubmission.model_validate_json(response.model_dump_json())
        if submission.command_id != command_id:
            raise ValueError("OPC submission belongs to another command")
        ack = await self.transport.report_submission(submission)
        ack = CommandReportAck.model_validate_json(ack.model_dump_json())
        submitted = ack.command_id == command_id
    except _UNCERTAIN:
        # No resubmission, including when delivery_possible=False or report ACK was lost.
        pass
try:
    response = await self.opc.get_command_result(
        CommandResultRequest(command_id=command_id)
    )
    result = _RESULT.validate_json(response.model_dump_json())
    if result.command_id != command_id:
        raise ValueError("OPC result belongs to another command")
    ack = await self.transport.report_result(result)
    ack = CommandReportAck.model_validate_json(ack.model_dump_json())
    if ack.command_id != command_id:
        raise ValueError("Journal receipt belongs to another command")
    return CommandWorkResult("reported", command_id, submitted, True)
except _UNCERTAIN:
    return CommandWorkResult("uncertain", command_id, submitted, False)
```

게이트웨이 명령 처리 코드 ↗ · 명령·사건 프로토콜 ↗

① 만료된 명령은 전달하지 않음

API가 이미 결과 불명으로 예약한 명령입니다. 늦게 PLC에 보내면 작업자가 모르는 동작이 됩니다.

② 전달 중 끊겨도 재전송 없음

PLC 접수 응답이나 보고 응답이 사라져도 같은 명령을 다시 제출하지 않습니다.

③ 결과는 항상 원래 ID로 조회

제출 여부와 관계없이 같은 command_id로 PLC 결과를 묻고, 다른 명령의 응답이면 거절합니다.

④ 확인 못 하면 uncertain

보고까지 확인되지 않으면 성공으로 기록하지 않고, 다음 주기에 같은 ID로 다시 조회합니다.

설계 의도: 같은 명령이 두 번 실행되는 것보다 결과를 잠시 모르는 편이 안전합니다. 판정은 PLC 원장이 하고, 운영 시스템은 그 결과를 원래 ID로 맞춰 봅니다.

작업지시에서 정산까지, 공개 화면으로 따라갈 수 있습니다

시연 순서와 근거 문서를 함께 제공합니다.

1 **로그인과 설비 전원**
역할별 권한, 원격 실행 설비 켜기

2 **작업지시**
공정 설정 선택 → 등록·배포

3 **운전**
불러오기 → 예열 → 생산 → 운전 중 설정 변경

4 **종료·정산**
일시정지·재개 → 종료 → 정산 → 같은 작업지시 재실행

5 **분석**
대시보드 · 보고서 · 트렌드 · SPC와 남은 검증 기준

GitHub 저장소 ↗ | 공개 데모 ↗

사용자 매뉴얼 ↗ | 개발 진행 상태 ↗



휴대폰 폭의 운전 제어 화면. 운전 중 실제 상태입니다.

자료 범위

화면: 공개 배포 · 2026-10-29 실제 운전 · 소스 59a1064

검증 수치: 저장소의 진행 기록과 수락 시험 대응표 기준

공개 화면의 설비는 임시 실행 환경이라 상시 켜져 있지 않습니다.

작성자: 전우석 · 개발 기간: 9일.

계정 비밀번호·DB 접속정보·인증서는 포함하지 않았습니다.